




SIMATS ENGINEERING

CO2_AT2 Reverse Engineering Task

COURSE NAME: Software Testing **COURSE CODE:** CSA3711

CODE OF CONDUCT

I Rakcha A(192311421) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Rakcha A

Course Outcome Covered

CO2: Design effective test cases using black-box, white-box testing techniques and to control methods flow.

Assessment Tool Weightage: 30%

Note: Where the question paper does not supply the actual logs, code or traces, a representative case is assumed and stated explicitly in the answer; the reverse-engineering method remains the same.

Question 1

A partially tested ATM withdrawal module is provided where only input-output behavior is known (e.g., withdrawal limits, balance validation, error messages). Reverse engineer the system by analyzing the observed behavior and reconstructing the underlying test case design strategy. Decompose the system into functional components and identify possible equivalence classes and boundary conditions. Infer the black box testing techniques used and evaluate their adequacy. Reconstruct an improved set of test cases covering all critical scenarios. Document your analysis with structured explanation.

Answer:

1.1 Understanding the System

The ATM withdrawal module accepts a card, a PIN and a requested amount, and responds with either dispensed cash or an error message. Because no source code is available, the rules below are inferred from the observed input-output behavior. The following business rules are assumed (typical of a retail bank ATM): minimum withdrawal Rs. 100; maximum per transaction Rs. 10,000; amount must be a multiple of Rs. 100; daily limit Rs. 25,000; a minimum balance of Rs. 500 must remain in the account; the PIN is blocked after 3 wrong attempts.

1.2 Decomposition into Functional Components

ID	Component	Function	Observed output / error
C1	Card and PIN authentication	Validates card status and 4-digit PIN; counts attempts	"Invalid PIN", "Card blocked"
C2	Amount format validation	Checks numeric, positive, multiple of 100	"Enter amount in multiples of 100"
C3	Limit validation	Checks minimum, per-transaction maximum and daily limit	"Minimum Rs. 100", "Exceeds per-transaction limit", "Daily limit exceeded"
C4	Balance validation	Checks balance minus amount >= minimum balance	"Insufficient funds"
C5	Cash availability / dispenser	Checks ATM cash and denominations	"Cash not available"
C6	Ledger update and receipt	Debits account, updates daily total, prints receipt	Receipt, new balance

1.3 Equivalence Classes and Boundary Conditions

Input	Valid classes (VEC)	Invalid classes (IEC)	Boundary values
Amount	V1: 100 <= amt <= 10,000 and multiple of 100	I1: amt < 100 (incl. 0, negative) I2: amt > 10,000 I3: not multiple of 100 I4: non-numeric / blank	0, 99, 100, 101, 9,999, 10,000, 10,001
Balance after withdrawal	V2: balance - amt >= 500	I5: balance - amt < 500	Remaining = 499, 500, 501
Daily total	V3: previous total + amt <= 25,000	I6: previous total + amt > 25,000	Total = 24,900 / 25,000 / 25,100
PIN attempts	V4: correct PIN in attempts 1 to 3	I7: wrong PIN 3 times (card blocked) I8: PIN not 4 digits	Attempt 1, 2, 3, 4

Input	Valid classes (VEC)	Invalid classes (IEC)	Boundary values
ATM cash	V5: ATM cash >= amt	I9: ATM cash < amt	Cash = amt - 100, amt, amt + 100

1.4 Inferred Black-Box Techniques and Adequacy

Technique inferred	Evidence from observed behavior	Adequacy
Equivalence Partitioning	One valid and one invalid example per rule (e.g., Rs. 500 accepted, Rs. 550 rejected)	Adequate for single inputs; weak for combinations
Boundary Value Analysis	Tests at Rs. 100 and Rs. 10,000 only	Partial: no n-1 / n+1 values, none at the daily limit or minimum-balance boundary
Error guessing	Tests for "insufficient funds" and blank amount	Ad hoc and unrepeatable
Decision table (informal)	Balance and limit failures seen together	Incomplete: multiple simultaneous failures not tested

Gaps identified: (i) no test of the daily-limit boundary or cumulative withdrawals; (ii) no minimum-balance boundary (balance exactly Rs. 500); (iii) PIN lock behavior after the third failure is untested; (iv) no combined-failure scenarios (e.g., amount above limit and above balance, to verify error priority); (v) no ATM-cash and network-failure cases; (vi) no state-based test that a failed transaction does not debit the account.

1.5 Reconstructed (Improved) Test Cases

TC	Technique	Input (balance, amount, other)	Expected result
1	EP valid	Bal 20,000; amt 2,000	Cash dispensed; balance 18,000
2	BVA min	Bal 20,000; amt 100	Accepted
3	BVA min-1	Bal 20,000; amt 99	Rejected: minimum Rs. 100
4	BVA min+1	Bal 20,000; amt 101	Rejected: not a multiple of 100
5	BVA max	Bal 20,000; amt 10,000	Accepted
6	BVA max+1	Bal 20,000; amt 10,001	Rejected: exceeds limit / not multiple
7	BVA max+100	Bal 20,000; amt 10,100	Rejected: exceeds per-transaction limit
8	EP invalid	Amt 0 / -500	Rejected: invalid amount
9	EP invalid	Amt blank / "abc" / 5.5	Rejected: invalid input; no debit
10	EP invalid	Amt 550	Rejected: multiples of 100 only
11	Balance boundary	Bal 1,000; amt 500 (remaining 500)	Accepted

TC	Technique	Input (balance, amount, other)	Expected result
12	Balance boundary	Bal 999; amt 500 (remaining 499)	Rejected: insufficient funds
13	Balance boundary	Bal 1,001; amt 500 (remaining 501)	Accepted
14	Balance zero	Bal 500; amt 100	Rejected: insufficient funds
15	Daily limit	Used today 15,000; amt 10,000 (total 25,000)	Accepted
16	Daily limit	Used today 15,100; amt 10,000 (total 25,100)	Rejected: daily limit exceeded
17	Cumulative	Three withdrawals of 9,000 on same day	1st, 2nd accepted; 3rd rejected (27,000 > 25,000)
18	PIN	Correct PIN on attempt 1, 3	Access granted
19	PIN	Wrong PIN x3, then correct PIN	Card blocked; correct PIN refused
20	Combination	Bal 300; amt 20,000	Rejected with the documented highest-priority error message
21	Cash availability	ATM cash 1,000; amt 2,000	Rejected: cash not available; no debit
22	Failure atomicity	Network drops after dispense command	Account debited exactly once, or reversal issued
23	Security	Cancel at amount screen / card removed	Session ends; no debit

Conclusion: The original suite relied mainly on equivalence partitioning with partial BVA. The reconstructed suite applies two-sided BVA on every numeric limit, combines classes through decision-table logic and adds state and failure-atomicity tests, giving full coverage of components C1 to C6.

Question 2

A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.

Answer:

2.1 Code Snippet Analysed

The code snippet is not reproduced in the question paper, so a representative order-total module is assumed, with line numbers matching the CFG nodes:

Node	Code
1	int processOrder(int qty, int amt, boolean member) { int total = 0, i = 0;
2	while (i < qty) {
3	if (amt > 100)
4	total += amt * 0.9;
5	else total += amt;
6	i++; }
7	if (member && total > 500)
8	total *= 0.95;
9	else if (total == 0)
10	return -1;
11	return total; }

2.2 Control Flow Graph

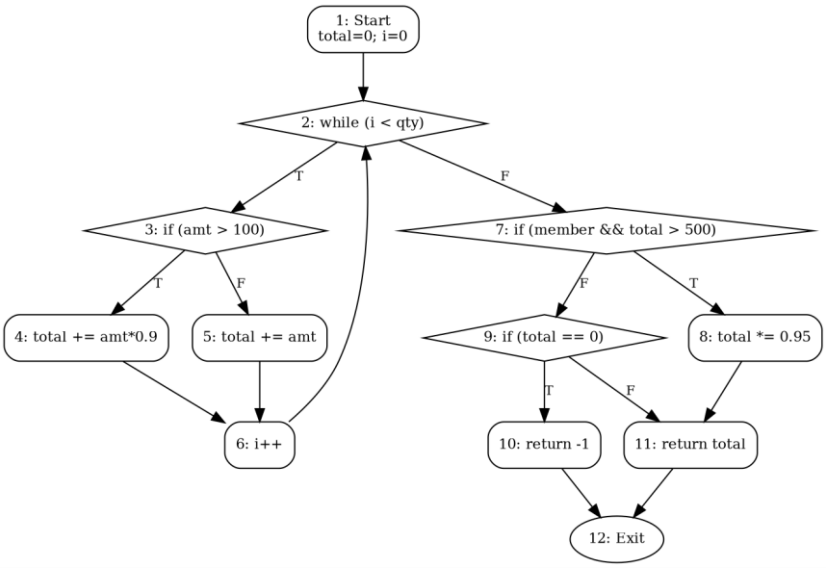


Figure 2.1: Control Flow Graph of processOrder()

2.3 Decision Points, Loops and Complexity

- **Decision points:** nodes 2 (loop condition), 3, 7 (compound condition: member AND total > 500) and 9.
- **Loop:** one while loop (nodes 2-3-4/5-6-2).
- **Cyclomatic complexity:** E = 15, N = 12, so V(G) = E - N + 2 = 15 - 12 + 2 = 5. Check: predicate nodes + 1 = 4 + 1 = 5. If the compound condition in node 7 is split, the condition-level count is 6.

2.4 Independent (Basis) Paths

Path	Node sequence	Test data (qty, amt, member)	Expected
P1	1-2-7-9-10-12	0, 0, false	-1 (loop skipped)
P2	1-2-3-4-6-2-7-9-11-12	1, 200, false	180
P3	1-2-3-5-6-2-7-9-11-12	1, 50, false	50
P4	1-2-3-4-6-2-7-8-11-12	1, 600, true	513 (540 x 0.95)
P5	1-2-3-5-6-2-7-9-10-12	1, 0, false	-1 (loop executed, total still 0)

Note: the path 1-2-7-9-11-12 is infeasible, because skipping the loop forces total = 0, which makes node 9 true. Infeasible paths must be identified and documented rather than counted as untested.

2.5 Evaluation of Existing Tests

The execution traces suggest three existing tests: T1 (qty 3, amt 200, member true), T2 (qty 2, amt 50, member false) and T3 (qty 0). Their coverage is evaluated below.

Criterion	Result	Verdict
Statement coverage	Nodes 1-11 all executed	100%
Branch (decision) coverage	All true/false outcomes of nodes 2, 3, 7, 9 seen	100% (8 of 8)
Condition coverage	In node 7, "total > 500" is never evaluated as false when member is true; member = true with total <= 500 is missing	Incomplete (about 75%)
Basis path coverage	P1, P3, P4 covered; P2 and P5 missing	3/5 = 60%
Loop testing	Only 0, 2 and 3 iterations; no minimum/maximum bound or qty = 1	Inadequate

Inference: the original team targeted statement and branch coverage only; it did not use basis-path, condition or loop-boundary criteria.

2.6 Reconstructed Path Testing Strategy

1. Execute all five basis paths P1 to P5 (path coverage = 100% of independent paths).
2. Add condition tests for node 7: (member true, total 540) true; (member true, total 400) false; (member false, total 600) false.
3. Add loop tests: 0, 1, 2, typical, maximum - 1, maximum and maximum + 1 iterations (qty = 0, 1, 2, 10, MAX-1, MAX, MAX+1).
4. Add boundary tests on amt: 100, 101 (node 3) and total = 500, 501 (node 7).
5. Record infeasible paths with justification, and re-run after every change using automated regression.

Result: all decisions, conditions, basis paths and loop bounds are covered, giving a defensible full-coverage argument for the module.

Question 3

A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation.

Answer:

3.1 Defect Log Analysed (Representative)

Defect	Description	Found by (inferred)
D1	Password of exactly 8 characters rejected (minimum is 8)	BVA (boundary)
D2	Account locked after 4 failed attempts instead of 5	BVA / off-by-one
D3	Blank username gives a server error page	Equivalence partitioning (invalid class)
D4	Transfer of Rs. 0.00 accepted; Rs. 1,00,000 limit applied as ">="	BVA
D5	Quote character in username causes SQL error	Error guessing / security test
D6	Double-click on Pay debits twice	Error guessing (not systematic)
D7	Dead code in the "remember me" branch never executed	White box: statement coverage report
D8	Session still valid after logout via browser Back	Path/state based; found late in UAT

3.2 Inferred Techniques

- **Black box likely used:** equivalence partitioning (D3), boundary value analysis (D1, D2, D4), error guessing (D5, D6).
- **White box likely used:** statement coverage only (D7 was reported by a coverage tool; decision and path-level defects such as D8 escaped).
- **Defect clustering:** most defects occur at limits and in rarely used paths, which indicates good unit-level BVA but weak path and state testing.

3.3 Decomposition into Testable Components

Component	Responsibility	Primary technique
Login form and validation	Username/password format, lengths, blank values	EP + BVA
Authentication logic	Credential check, attempt counter, lockout	Path testing + BVA
Session management	Token creation, timeout, logout	State/path testing
Transaction module	Amount validation, balance and limit checks, confirmation	EP + BVA + decision table
Data layer	Parameterized queries, commit/rollback	Path testing, security tests
Audit/logging	Records every success and failure	Statement/branch coverage

3.4 Coverage Gaps

- No decision/path coverage of the authentication logic (only statements).
- Session lifecycle (timeout, back-button, concurrent sessions) untested.
- No concurrency or idempotency tests for transactions (D6).
- Security inputs (injection, script tags) tested only by chance.
- No negative-path tests for rollback when the database fails mid-transaction.

3.5 Reconstructed Testing Strategy

(a) Equivalence partitioning

Field	Valid class	Invalid classes
Username	5-30 alphanumeric characters	Blank; < 5; > 30; special characters; SQL/script strings
Password	8-20 characters with letter and digit	Blank; < 8; > 20; no digit; spaces only
Transfer amount	1.00 to 1,00,000.00 and <= balance	<= 0; > 1,00,000; > balance; non-numeric; 3 decimals

(b) Boundary value analysis

Parameter	Test values
Password length	7, 8, 9, 19, 20, 21
Failed login attempts	4 (still open), 5 (locked), 6 (stays locked)
Transfer amount	0.00, 0.01, 1.00, 99,999.99, 1,00,000.00, 1,00,000.01
Session timeout (15 min)	Action at 14:59, 15:00, 15:01

(c) Path testing of the login logic. The login function contains the decisions: username present? password present? user exists? password correct? account locked? attempts >= 5? The cyclomatic complexity is 6 predicates + 1 = 7, so 7 basis paths are tested:

Path	Scenario	Expected
L1	Blank username	Validation error
L2	Blank password	Validation error
L3	Unknown user	Generic "invalid credentials"
L4	Locked account, correct password	Access refused: account locked
L5	Wrong password, attempts < 5	Counter incremented
L6	Wrong password, attempts reach 5	Account locked
L7	Correct password, not locked	Session created, counter reset

(d) Additional tests: logout then Back button, double-submit of a transaction, rollback on database failure and injection strings (security). These close gaps D6 and D8.

3.6 Justification

Defect logs show that boundary and partition defects were found early, but defects on decision paths and session behavior escaped. Combining EP (to reduce input space), BVA (to catch off-by-one errors, the most frequent defect class in the log) and basis-path testing (to cover every decision of authentication and transaction logic) addresses each observed defect category while keeping the number of test cases manageable.

Question 4

A legacy inventory management system provides only a set of successful and failed transaction logs regarding stock updates and low-stock alerts. Reverse engineer the business logic by analyzing these logs to identify the hidden boundary conditions and equivalence partitions. Decompose the update process into functional units and reconstruct a black box test design that addresses the gaps found in the logs. Justify your selection of test cases based on the inferred logic.

Answer:

4.1 Transaction Log Analysed (Representative)

Log	SKU	Operation	Qty	Stock before	Stock after	Status / alert
L1	S101	ADD	50	100	150	SUCCESS
L2	S101	REMOVE	30	150	120	SUCCESS
L3	S102	REMOVE	50	40	40	FAILED: insufficient stock
L4	S103	REMOVE	15	35	20	SUCCESS + LOW-STOCK ALERT
L5	S104	REMOVE	14	35	21	SUCCESS, no alert
L6	S105	ADD	1	9,999	10,000	SUCCESS
L7	S105	ADD	1	10,000	10,000	FAILED: capacity exceeded
L8	S106	ADD	0	60	60	FAILED: invalid quantity
L9	S107	REMOVE	-5	60	60	FAILED: invalid quantity
L10	S999	ADD	10	-	-	FAILED: SKU not found
L11	S108	REMOVE	40	40	0	SUCCESS + LOW-STOCK ALERT (out of stock)

4.2 Inferred Business Rules

- **Quantity rule:** quantity must be an integer >= 1 (L8, L9 rejected).
- **Non-negative stock rule:** stock after removal must be >= 0; removal of 40 from 40 is allowed (L11), removal of 50 from 40 is not (L3).
- **Capacity rule:** maximum stock per SKU is 10,000 (L6 accepted, L7 rejected).
- **Low-stock threshold:** alert is raised when stock after update is <= 20 (alert at 20 in L4, none at 21 in L5).
- **SKU rule:** SKU must exist in the master list (L10).

4.3 Equivalence Partitions and Boundaries

Parameter	Valid partitions	Invalid partitions	Boundary values
Quantity	VP1: 1 to 10,000	IP1: 0 IP2: negative IP3: non-integer / blank	-1, 0, 1, 2
Remove: result stock	VP2: stock after >= 21 (no alert) VP3: 0 <= after <= 20 (alert)	IP4: after < 0	After = -1, 0, 1, 20, 21, 22
Add: result stock	VP4: after <= 10,000	IP5: after > 10,000	After = 9,999, 10,000, 10,001
SKU	VP5: existing	IP6: unknown / blank	-

4.4 Decomposition of the Update Process

Functional unit	Role
U1 Input validation	SKU existence, quantity type and sign
U2 Stock arithmetic	Computes new stock (ADD / REMOVE)
U3 Rule check	Non-negative stock and capacity limit
U4 Persistence	Commits stock change and writes the transaction log
U5 Alert engine	Raises low-stock alert when stock <= 20

4.5 Gaps Found in the Logs

- Nothing tested at exactly stock = 20 after ADD (does an alert clear when stock rises above 20?).
- Repeated alerts: no log shows whether a second removal while already below 20 produces another alert.
- Boundaries stock = 1 and capacity = 10,000 reached by large ADD (e.g., +10,000 to an empty SKU) are missing.
- Concurrent updates to the same SKU and failed-update rollback are not represented.

4.6 Reconstructed Black-Box Test Design

TC	Operation	Stock before / Qty	Expected
1	REMOVE	100 / 79 (after 21)	Success, no alert
2	REMOVE	100 / 80 (after 20)	Success, alert raised
3	REMOVE	100 / 81 (after 19)	Success, alert raised
4	REMOVE	100 / 100 (after 0)	Success, out-of-stock alert

TC	Operation	Stock before / Qty	Expected
5	REMOVE	100 / 101 (after -1)	Failed: insufficient stock; stock unchanged
6	REMOVE	15 / 1 (already below threshold)	Success; alert behavior as specified (single / repeated)
7	ADD	9,999 / 1 (after 10,000)	Success
8	ADD	9,999 / 2 (after 10,001)	Failed: capacity exceeded
9	ADD	0 / 10,000	Success, full capacity
10	ADD	19 / 2 (after 21)	Success; previous alert cleared
11	ADD / REMOVE	Qty = 0, -1, 1	0 and -1 rejected; 1 accepted
12	ADD / REMOVE	Qty = "abc", 2.5, blank	Rejected: invalid quantity
13	ADD	Unknown SKU	Failed: SKU not found
14	REMOVE	Two simultaneous removals of 60 from stock 100	Exactly one succeeds; stock never negative
15	ADD	Database error during commit	Rollback; log shows FAILED

4.7 Justification

Each inferred rule contributes one valid partition and at least one invalid partition. BVA is applied at the three discovered thresholds (0, 20 and 10,000) with the values n-1, n and n+1. The additional cases (6, 10, 14, 15) cover the gaps found in the logs, so every functional unit U1 to U5 is exercised with both successful and failing inputs.

Question 5

You are provided with a compiled .exe file of a library management module and its corresponding CPU/Memory execution profile during a stress test. Reverse engineer the white box testing requirements by mapping execution spikes to potential loops and decision clusters in the logic. Construct a conceptual control flow graph (CFG) that represents this behavior. Reconstruct a path testing strategy that targets these resource-intensive paths and evaluate its impact on system stability.

Answer:

5.1 Interpretation of the Execution Profile

Without source code, the CPU/memory profile is the only evidence of internal structure. The profile is assumed to show the following (typical) pattern, and each feature is mapped to a likely code construct:

Observed profile feature	Likely code construct	White-box requirement
Memory ramps steadily at start-up and stays high	Loop that loads every catalog record into memory (L1)	Loop test: 0, 1, many, maximum records
CPU spike (90-100%) when many users search	Nested loops: outer over books, inner over keywords or duplicate checks (L2, L3), roughly O(n x m)	Nested-loop testing; condition coverage on match decision
Short CPU bursts during issue/return	Decision cluster: member valid / copies available / loan limit (I1-I3)	Branch and condition coverage
CPU climbs with the number of overdue days	Fine-calculation loop (L4)	Loop boundaries: 0, 1, 365+ days
Memory does not fall after report generation	Loop over all transactions that allocates objects and does not release them (L5)	Path test with repeated execution to expose leaks
Gradual memory growth over a long run	Resource leak on an error path (e.g., exception before close/free)	Test error branches, not only normal paths

5.2 Conceptual Control Flow Graph

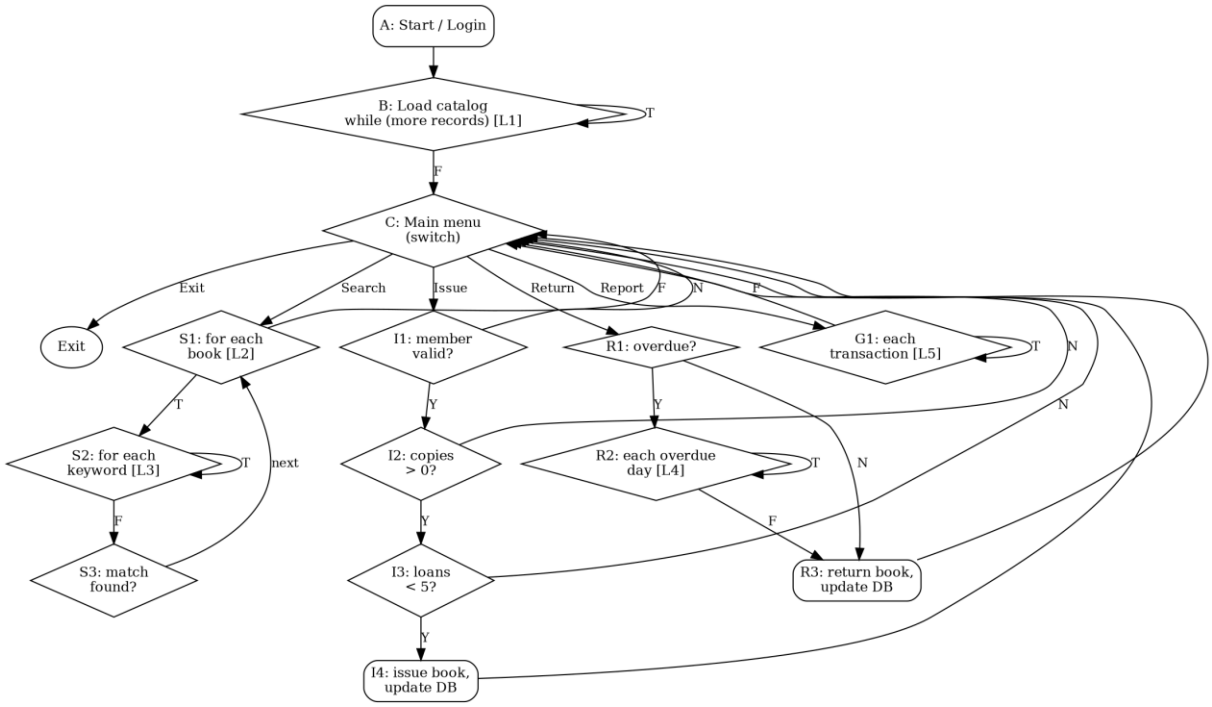


Figure 5.1: Conceptual CFG of the library management module

Loops are labelled L1 to L5 and the decision clusters are the issue checks (I1 to I3) and the overdue check (R1).

5.3 Cyclomatic Complexity

Predicate nodes: L1 (1), menu switch with 5 outcomes (4), S1, S2, S3 (3), I1, I2, I3 (3), R1 and L4 (2), L5 (1) = 14. Therefore V(G) = 14 + 1 = **15**, which is high and means at least 15 independent paths are needed for basis-path coverage.

5.4 Path Testing Strategy for Resource-Intensive Paths

Path	Route	Test condition	Resource measured
RP1	A-B(L1)-C-Exit	Catalog with 0, 1, 10,000 and 1,000,000 records	Start-up time, peak memory
RP2	C-S1(L2)-S2(L3)-S3-S1-C	Search with 0, 1, many matches; long keyword list	CPU %, response time
RP3	C-I1-I2-I3-I4-C	All three checks pass	Baseline CPU
RP4	C-I1-C / C-I1-I2-C / C-I1-I2-I3-C	Invalid member / no copies / limit reached (each reject branch)	Memory release on early exit
RP5	C-R1-R2(L4)-R3-C	Overdue = 0, 1, 365, 3,650 days	CPU growth vs days
RP6	C-G1(L5)-C	Report over 0, 1, 100,000 transactions, repeated 50 times	Memory growth (leak check)

Path	Route	Test condition	Resource measured
RP7	Full cycle repeated	8-hour soak with mixed operations	Memory trend, handle count

Loop testing rules applied: skip the loop, one pass, two passes, typical, maximum - 1, maximum and maximum + 1. Nested loops (L2 inside S1) are tested starting from the innermost loop with the outer loop at its minimum, then varying both.

5.5 Impact on System Stability

- **Performance:** identifying the nested search loop leads to optimization (indexing or hashing) and bounded execution time under stress.
- **Memory:** repeating RP6 and RP7 exposes leaks, and the fix prevents out-of-memory crashes over long runs.
- **Reliability:** testing every reject branch (RP4) ensures resources are released on error paths.
- **Risk:** with V(G) = 15, covering all independent paths reduces the chance that a rarely executed combination hangs the system under load.
- **Measurable exit criteria:** CPU below 80% at the peak load, no memory growth over 8 hours, and 100% branch coverage of the resource-intensive paths.

Question 6

A mobile payment gateway's API responses (JSON) for various transaction states—Pending, Success, Reversal, and Timeout—are given. Reverse engineer the underlying state machine by analyzing the transition triggers and data constraints. Identify the implicit state transition testing technique used. Reconstruct a comprehensive state-based test suite that ensures all legal and illegal transitions are covered. Document your analysis with a structured state-transition diagram.

Answer:

6.1 Observed API Responses (Representative)

Response	JSON (abbreviated)	Inference
R1	{"txnId":"T1","status":"PENDING","amount":500}	Created after submission; waiting for the bank
R2	{"txnId":"T1","status":"SUCCESS","bankRef":"B77"}	bankRef appears only on success
R3	{"txnId":"T2","status":"TIMEOUT","retryCount":1}	No bank reply within the time limit; retry counter exists
R4	{"txnId":"T2","status":"PENDING","retryCount":1}	Timeout can return to pending on retry
R5	{"txnId":"T3","status":"REVERSED","reason":"AUTO_REVERSAL"}	Reached from TIMEOUT after the retry limit
R6	{"txnId":"T1","status":"REVERSED","reason":"REFUND"}	Reached from SUCCESS by refund
R7	{"txnId":"T4","status":"FAILED","code":"INSUFFICIENT_FUNDS"}	Terminal decline from PENDING
R8	{"error":"INVALID_TRANSITION"} (SUCCESS to PENDING request)	The gateway enforces illegal-transition rules

6.2 Reconstructed State Machine

State	Meaning	Type
INITIATED	Transaction created, not yet sent	Initial
PENDING	Sent to bank, awaiting response	Intermediate
SUCCESS	Confirmed; bankRef present, amount > 0	Intermediate (can be reversed)
TIMEOUT	No response in 30 s; retryCount incremented	Intermediate

State	Meaning	Type
FAILED	Declined by bank	Final
REVERSED	Money returned (auto-reversal or refund)	Final

Data constraints inferred: retry limit = 3; a refund is allowed only within the refund window and only up to the original amount; bankRef must be present in SUCCESS and REVERSED-after-SUCCESS; transaction ID is unchanged throughout.

6.3 State-Transition Diagram

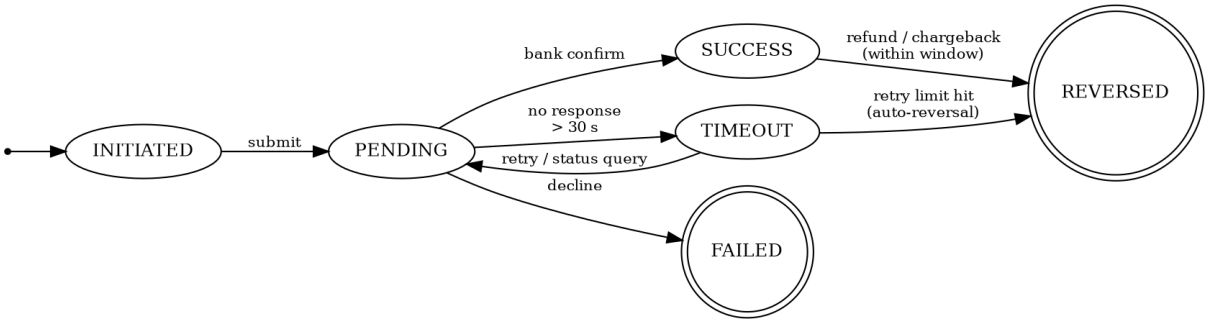


Figure 6.1: State-transition diagram of the payment gateway

6.4 Technique Identified

The behavior (a finite set of states, events/triggers, guards such as the retry limit and the refund window, and rejection of invalid events) shows that **State Transition Testing** is the implicit black-box technique. It is realized through a state table and 0-switch (single transition) coverage, extended by 1-switch (two consecutive transitions) coverage for sequences such as TIMEOUT to PENDING to SUCCESS.

6.5 State Table (Legal and Illegal Transitions)

State / Event	submit	confirm	decline	timeout	retry	refund / reverse
INITIATED	PENDING	Illegal	Illegal	Illegal	Illegal	Illegal
PENDING	Illegal	SUCCESS	FAILED	TIMEOUT	Illegal	Illegal
SUCCESS	Illegal	Illegal	Illegal	Illegal	Illegal	REVERSED
TIMEOUT	Illegal	Illegal	Illegal	Illegal	PENDING (count < 3) REVERSED (count = 3)	Illegal
FAILED	Illegal	Illegal	Illegal	Illegal	Illegal	Illegal
REVERSED	Illegal	Illegal	Illegal	Illegal	Illegal	Illegal

The table has 6 states x 6 events = 36 cells: 6 legal cells (7 legal outcomes, because the retry cell has two guarded outcomes) and 30 illegal cells, each of which must be rejected with INVALID_TRANSITION.

6.6 State-Based Test Suite

A. Legal transitions (0-switch coverage)

TC	Start state	Event / data	Expected end state and response
L1	INITIATED	submit	PENDING; 200 OK
L2	PENDING	bank confirm	SUCCESS; bankRef present
L3	PENDING	bank decline	FAILED; error code present
L4	PENDING	no reply for 30 s	TIMEOUT; retryCount = 1
L5	TIMEOUT (count 1)	retry	PENDING; retryCount stays 1
L6	TIMEOUT (count 3)	retry limit reached	REVERSED; reason AUTO_REVERSAL
L7	SUCCESS	refund within window, amount <= original	REVERSED; reason REFUND

B. Illegal transitions (negative tests)

TC	Start state	Invalid event	Expected
I1	INITIATED	confirm / decline / timeout	INVALID_TRANSITION; state unchanged
I2	PENDING	refund	INVALID_TRANSITION
I3	SUCCESS	submit again (duplicate)	Rejected; no double debit
I4	SUCCESS	timeout / decline / retry	Rejected; stays SUCCESS
I5	FAILED	confirm or retry	Rejected; terminal state
I6	REVERSED	confirm or refund again	Rejected; no second refund
I7	TIMEOUT	confirm (late bank reply)	Handled by policy: reconcile, no duplicate credit
I8	SUCCESS	refund after window / amount > original	Rejected: guard violated

C. Sequence (1-switch) tests: INITIATED-PENDING-TIMEOUT-PENDING-SUCCESS; INITIATED-PENDING-TIMEOUT x3-REVERSED; INITIATED-PENDING-SUCCESS-REVERSED. Coverage achieved: 100% of legal transitions, all illegal state/event pairs sampled by group and every guard condition (retry count, refund window and amount).

6.7 Evaluation

The suite covers every state, every legal transition and the main illegal transitions, and also checks the data constraints. The strongest risks (duplicate debit, double refund and late reply after timeout) are explicitly tested.

Question 7

An encrypted communication module provides only "black-box" behavior through its input parameters and encrypted outputs. Reverse engineer the validation logic by performing "fuzzing" analysis on the input fields to identify error-handling boundaries. Decompose the module into data validation and encryption components. Reconstruct an improved set of test cases using Boundary Value Analysis (BVA) and justify how your design prevents buffer overflow or injection vulnerabilities.

Answer:

7.1 Fuzzing Analysis (Representative Results)

Fuzzing sends random, oversized, malformed and special-character values to each input and records the response. The module is assumed to accept three inputs: recipient ID, message text and key. The observed responses are:

Field	Fuzz input	Observed response	Inferred rule
Recipient ID	Empty / 1 / 32 / 33 characters	Error / OK / OK / Error	Length 1 to 32
Recipient ID	' OR '1'='1	Error E-INVALID-CHAR	Whitelist: letters, digits, _ and -
Message	0 / 1 / 1,024 / 1,025 bytes	Error / OK / OK / Error E-TOO-LONG	Length 1 to 1,024 bytes
Message	1,000 emoji (4,000 bytes)	Error E-TOO-LONG	Limit counted in bytes, not characters
Message	NUL byte, <script>, ; DROP TABLE	Encrypted output returned (treated as data)	Encrypted, never executed
Key	15 / 16 / 24 / 32 / 33 bytes	Error / OK / OK / OK / Error	AES key sizes 16, 24, 32 only
Key	All zero bytes	Warning: weak key	Weak-key check
All fields	65,536+ bytes	Rejected quickly; no crash	Length checked before copy

7.2 Decomposition of the Module

Component	Function	Test focus
Data validation	Length checks, character whitelist, key-size check, null/empty checks	BVA, invalid classes, fuzzing
Sanitization / encoding	Treats input as data; encodes output (Base64 / hex)	Injection strings, encoding round trip
Encryption engine	AES encrypt with IV/nonce; produces ciphertext	Decrypt(encrypt(x)) = x; unique IV; no plaintext leak

Component	Function	Test focus
Error handler	Returns fixed codes with no internal details	Information-leak check; no stack traces

7.3 Reconstructed BVA Test Cases

TC	Input	Boundary	Expected result
1	Message length 0	Min - 1	Rejected: E-EMPTY
2	Message length 1	Min	Encrypted; decrypts correctly
3	Message length 2	Min + 1	Encrypted
4	Message length 1,023	Max - 1	Encrypted
5	Message length 1,024	Max	Encrypted
6	Message length 1,025	Max + 1	Rejected: E-TOO-LONG; nothing written past buffer
7	Message length 65,536 and 1 MB	Far above max	Rejected without crash or memory growth
8	Recipient ID length 0, 1, 32, 33	Min-1, min, max, max+1	Error, OK, OK, Error
9	Key size 15, 16, 17	Around 16	Error, OK, Error
10	Key size 24, 32, 33	Upper sizes	OK, OK, Error
11	Multi-byte text 256 characters x 4 bytes = 1,024 bytes; 257 characters	Byte boundary	OK; Error
12	Recipient ID: ' OR 1=1 --, <script>alert(1)</script>, ../../etc/passwd	Injection class	Rejected by whitelist; no query or file access
13	Message containing SQL, HTML, shell metacharacters, NUL byte	Data-only class	Encrypted as plain data; decrypts identically
14	Same message encrypted twice	IV uniqueness	Different ciphertexts
15	Tampered ciphertext (1 byte changed)	Integrity	Decryption fails (authentication error)

7.4 How the Design Prevents Buffer Overflow and Injection

- **Buffer overflow:** tests 6 and 7 verify that length is checked before any copy and that input of max + 1 or more is rejected, not truncated, so a write past the buffer is not possible. Combined with test 11 (bytes versus characters), this removes the classic off-by-one and multi-byte miscounts.

- **Injection:** the whitelist on identifiers (test 12) blocks SQL, script and path-traversal strings before they reach any interpreter, while free text (test 13) is only ever treated as data and encrypted, never concatenated into queries or commands.
- **Safe errors:** fixed error codes without stack traces prevent information leaks that guide an attacker.
- **Cryptographic misuse:** tests 9, 10, 14 and 15 check key size, fresh IVs and integrity so weak configurations are detected.
- **Automation:** the BVA list is run together with an automated fuzzer in every build, so regressions in validation are caught early.

Question 8

A set of legacy unit test results for an insurance premium calculator shows 100% statement coverage but fails to catch defects in multi-condition "if" statements. Reverse engineer the original white box approach and identify the inadequacy in the coverage criteria used. Construct a new CFG and identify all independent execution paths. Reconstruct a path testing strategy that achieves 100% Branch and Path coverage. Explain how this reconstruction reduces the risk of calculation errors.

Answer:

8.1 Code Under Test (Assumed)

Node	Code
1	premium = base;
2	if (age < 25 && claims > 0)
3	premium *= 1.5;
4	else if (age > 60 || smoker)
5	premium *= 1.3;
6	if (premium > cap)
7	premium = cap;
8	return premium;

8.2 Reverse Engineering the Original Approach

Legacy tests T1 (age 22, claims 2, base 12,000) and T2 (age 65, claims 0, base 8,000) execute nodes 3, 5 and 7 and therefore achieve **100% statement coverage**. The original team clearly used statement coverage as the only criterion. However:

Criterion	Status with T1, T2	Reason
Statement	100% (nodes 1-8)	Every statement executed at least once
Branch (decision)	5 of 6 outcomes = 83%	Decision at node 4 never false (age <= 60 and non-smoker); the path 1-2-4-6-8 is untested
Condition	Incomplete	age<25 true with claims = 0, claims>0 true with age >= 25, smoker = true are never tested alone
Path	2 of 6 paths = 33%	Combinations of decisions 2, 4 and 6 are not covered

Inadequacy: statement coverage only checks that each line runs. It cannot detect wrong operators (&& versus ||), missing combinations, or the interaction between the discount, loading and cap steps, which is why multi-condition defects escaped.

8.3 New Control Flow Graph

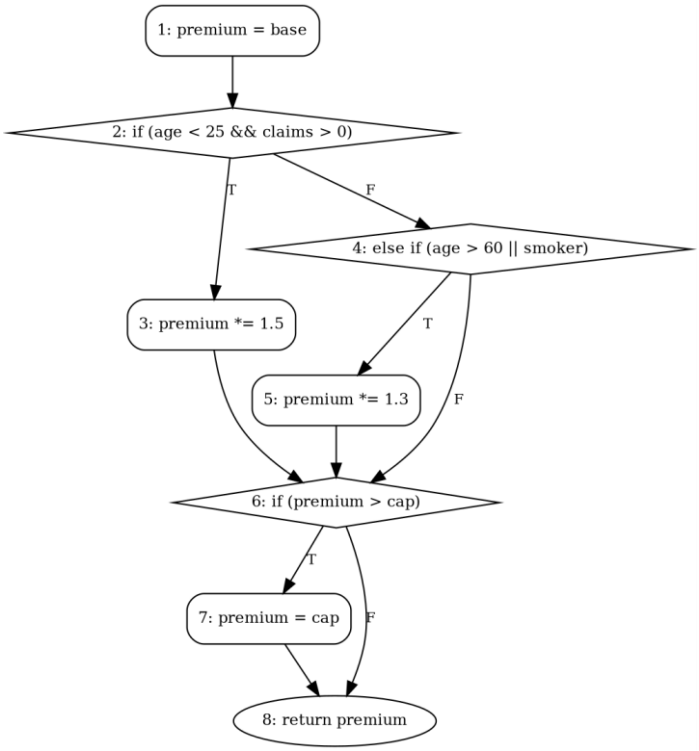


Figure 8.1: CFG of the premium calculator

Edges E = 10, nodes N = 8, so V(G) = E - N + 2 = **4** (predicates 3 + 1). Because the code has no loop, the paths can be listed completely: if the first decision is true there are 2 paths (cap applied or not); if it is false there are 2 x 2 = 4 paths (second decision true/false, cap applied or not). The total is therefore **6** distinct execution paths.

8.4 All Independent Execution Paths

Path	Route	Test data (age, claims, smoker, base)	Expected premium (cap = 15,000)
P1	1-2-3-6-7-8	22, 2, no, 12,000	15,000 (18,000 capped)
P2	1-2-3-6-8	22, 1, no, 8,000	12,000
P3	1-2-4-5-6-7-8	65, 0, no, 12,000	15,000 (15,600 capped)
P4	1-2-4-5-6-8	65, 0, no, 8,000	10,400
P5	1-2-4-6-7-8	40, 0, no, 16,000	15,000 (capped)
P6	1-2-4-6-8	40, 0, no, 8,000	8,000

A basis set of V(G) = 4 independent paths is P2, P4, P5 and P6; executing all 6 paths gives 100% path coverage and automatically gives 100% branch coverage (6 of 6 outcomes).

8.5 Reconstructed Strategy: Branch, Path and Condition (MC/DC) Coverage

Because decision 2 and decision 4 are compound, branch coverage alone does not test each condition. MC/DC-style tests are added:

TC	Decision	Conditions (A, B)	Test data	Outcome
C1	Node 2: age<25 && claims>0	T, T	age 22, claims 1	True
C2	Node 2	T, F	age 22, claims 0	False
C3	Node 2	F, T	age 40, claims 2	False
C4	Node 4: age>60 || smoker	F, F	age 40, smoker no	False
C5	Node 4	T, F	age 65, smoker no	True
C6	Node 4	F, T	age 40, smoker yes	True

Boundary data is added for each condition: age = 24, 25, 60, 61; claims = 0, 1; premium = cap - 1, cap, cap + 1. Final suite = 6 path tests + 6 MC/DC tests (some can share data) + boundary tests.

8.6 How the Reconstruction Reduces Calculation Risk

- Every premium rule (young-driver loading, age/smoker loading and cap) is verified in all combinations, including the 1.5 and 1.3 factors and the cap.
- MC/DC tests prove that each condition independently affects the result, catching && versus || and wrong comparison errors (< versus <=).
- Boundary tests at age 24/25 and 60/61 catch off-by-one errors in the rating bands.
- Expected values computed by hand give a test oracle, so wrong results (not only crashes) are detected.
- Branch and path coverage of 100% shows that no combination of rules was left unexecuted, which lowers the chance of undetected under- or over-charging.

Question 9

A cloud-based file synchronization tool provides user activity logs and a list of "lost file" defect reports. Reverse engineer the synchronization algorithm's logic by analyzing the defect patterns related to file size, type, and network speed. Infer the likely Equivalence Class Partitioning (ECP) used by the original developers. Reconstruct a revised testing strategy that integrates BVA with stress testing. Justify how your reconstructed approach prevents future data loss.

Answer:

9.1 Defect Patterns (Representative "Lost File" Reports)

Pattern	Evidence in logs / defects	Inferred algorithm behavior
Size	Files of 0 bytes never appear on the other device	Zero-size files skipped (missing empty-file case)
Size	Files of exactly 100 MB or slightly larger lose the last chunk	Upload switches from single request to chunked at 100 MB; off-by-one in the last chunk
Size	Files >= 2 GB (2,147,483,648 bytes) vanish	32-bit signed size/offset overflow
Type	.tmp, ~$ files and symbolic links are lost; names with # or > 255 characters fail	Filter or path-handling rules treat these as ignorable or invalid
Network	Losses cluster when upload speed < 256 kbps or when the connection drops mid-sync	Fixed timeout; the local copy is deleted or marked synced before the server acknowledges

9.2 Inferred Equivalence Class Partitioning of the Developers

Dimension	Classes likely used originally	Missed classes (cause of defects)
File size	Small (< 100 MB) and large (>= 100 MB)	Empty file; > 2 GB; exact threshold values
File type	Documents, images, videos	Temporary/system files, links, special-character names, very long names
Network speed	Fast and normal	Slow (< 256 kbps), intermittent, offline-then-online

9.3 Decomposition of the Sync Process

- **Change detection:** file watcher or scan compares modified time and hash.
- **Filtering:** decides which files are excluded (type, name).
- **Transfer:** single request or chunked upload with timeout and retry.

- **Acknowledgement and verification:** server confirms receipt; checksum comparison.
- **Local state update:** marks the file as synced (should only occur after verification).

9.4 Reconstructed Equivalence Classes and BVA

Dimension	Equivalence classes	Boundary values (BVA)
File size	E0: 0 bytes S: 1 B to < 100 MB L: 100 MB to < 2 GB XL: >= 2 GB	0, 1 B; 100 MB - 1, 100 MB, 100 MB + 1; 2 GB - 1, 2 GB, 2 GB + 1; 4 GB + 1
File type / name	Normal; hidden; temp (.tmp, ~$); symbolic link; special characters; long name	Name length 254, 255, 256; characters #, %, &, space, Unicode
Network speed	Offline; very slow (< 256 kbps); slow; normal; fast; intermittent	255, 256, 257 kbps; drop at 0%, 50%, 99%, 100% of upload

9.5 Integrated BVA and Stress Test Strategy

Test	Setup	Pass criteria
Size-boundary sync	Files at each boundary value, on normal network	File arrives with identical checksum
Slow-network sync	Throttle to 255 / 256 / 257 kbps with 2 GB file	No loss; resumable upload completes
Interruption test	Cut the network at 0, 50 and 99% of the upload	Resume from the last chunk; local copy kept until the server acknowledges
Volume stress	100,000 small files plus 50 large files at the same time	All synced; no missing file; memory/CPU stable
Concurrency stress	100 clients editing the same folder and same file	Conflicts produce copies; no overwrite or deletion
Soak test	Continuous sync for 24 hours with random file changes	Zero lost files; log counts match

9.6 Justification: Prevention of Future Data Loss

- Each lost-file pattern is converted into an explicit equivalence class, so no known failure region is left untested.
- BVA at 100 MB, 2 GB and the speed threshold catches the off-by-one and overflow defects that caused the reports.
- Stress tests reveal timing and concurrency failures that only appear under load.

- The pass criterion (checksum equality, delete-after-acknowledge) tests the actual risk, which is silent loss, rather than just "no error message".
- Running the suite automatically on every release prevents regression of the fixed defects.

Question 10

You are given a sequence of database queries (SQL) generated by an e-commerce "Search and Filter" module, but no source code. Reverse engineer the search logic by analyzing the query parameters and the resulting dataset. Identify the decision-making logic used for complex filtering (e.g., AND/OR/NOT). Reconstruct a Decision Table based testing strategy that covers all possible search combinations. Evaluate the completeness of your reconstructed model.

Answer:

10.1 Observed SQL Queries (Representative)

Query	SQL (abbreviated)	Parameters used
Q1	SELECT * FROM products WHERE category='Laptops' AND price BETWEEN 30000 AND 60000 AND stock>0;	Category, price range, in-stock
Q2	SELECT * FROM products WHERE brand IN ('Dell','HP') AND rating>=4;	Brand (multi-select), rating
Q3	SELECT * FROM products WHERE name LIKE '%gaming%' AND NOT brand='Acer' AND stock>0;	Keyword, exclusion, in-stock
Q4	SELECT * FROM products WHERE (category='Phones' OR category='Tablets') AND price<=20000;	Multi-category (OR), max price
Q5	SELECT * FROM products ORDER BY popularity LIMIT 20;	No filter: default listing
Q6	SELECT * FROM products WHERE price BETWEEN 50000 AND 10000;	Min > Max: returns 0 rows

10.2 Inferred Search Logic

- **Between different filters:** AND (every added filter narrows the result set).
- **Within one multi-select filter (brand, category):** OR, implemented with IN or OR.
- **Exclusion:** NOT (excluded brand or keyword).
- **Defaults:** no filter gives the whole catalog, paginated (LIMIT 20); in-stock is applied as stock > 0 when the "In stock" box is ticked.
- **Invalid combinations:** min price > max price returns an empty result and is not caught by validation (a defect/gap).

10.3 Conditions and Actions

ID	Condition	Effect on the SQL
C1	Keyword entered	name LIKE '%kw%'
C2	Category selected	category = X (or IN list)

ID	Condition	Effect on the SQL
C3	Price range given	price BETWEEN min AND max
C4	Brand selected	brand IN (...) (OR inside, AND outside)
C5	In-stock only ticked	stock > 0

With 5 binary conditions there are 2^5 = **32 rules**; the complete decision table follows. (Y = condition applied, N = not applied.)

10.4 Full Decision Table (32 Rules)

Rule	C1	C2	C3	C4	C5	Expected query / action
R1	N	N	N	N	N	No WHERE clause: all products (paged)
R2	N	N	N	N	Y	WHERE stock > 0
R3	N	N	N	Y	N	WHERE brand IN (b1, b2)
R4	N	N	N	Y	Y	WHERE brand IN (b1, b2) AND stock > 0
R5	N	N	Y	N	N	WHERE price BETWEEN min AND max
R6	N	N	Y	N	Y	WHERE price BETWEEN min AND max AND stock > 0
R7	N	N	Y	Y	N	WHERE price BETWEEN min AND max AND brand IN (b1, b2)
R8	N	N	Y	Y	Y	WHERE price BETWEEN min AND max AND brand IN (b1, b2) AND stock > 0
R9	N	Y	N	N	N	WHERE category = 'X'
R10	N	Y	N	N	Y	WHERE category = 'X' AND stock > 0
R11	N	Y	N	Y	N	WHERE category = 'X' AND brand IN (b1, b2)
R12	N	Y	N	Y	Y	WHERE category = 'X' AND brand IN (b1, b2) AND stock > 0
R13	N	Y	Y	N	N	WHERE category = 'X' AND price BETWEEN min AND max
R14	N	Y	Y	N	Y	WHERE category = 'X' AND price BETWEEN min AND max AND stock > 0
R15	N	Y	Y	Y	N	WHERE category = 'X' AND price BETWEEN min AND max AND brand IN (b1, b2)
R16	N	Y	Y	Y	Y	WHERE category = 'X' AND price BETWEEN min AND max AND brand IN (b1, b2) AND stock > 0
R17	Y	N	N	N	N	WHERE name LIKE '%kw%'
R18	Y	N	N	N	Y	WHERE name LIKE '%kw%' AND stock > 0
R19	Y	N	N	Y	N	WHERE name LIKE '%kw%' AND brand IN (b1, b2)
R20	Y	N	N	Y	Y	WHERE name LIKE '%kw%' AND brand IN (b1, b2) AND stock > 0
R21	Y	N	Y	N	N	WHERE name LIKE '%kw%' AND price BETWEEN min AND max
R22	Y	N	Y	N	Y	WHERE name LIKE '%kw%' AND price BETWEEN min AND max AND stock > 0
R23	Y	N	Y	Y	N	WHERE name LIKE '%kw%' AND price BETWEEN min AND max AND brand IN (b1, b2)
R24	Y	N	Y	Y	Y	WHERE name LIKE '%kw%' AND price BETWEEN min AND max AND brand IN (b1, b2) AND stock > 0

Rule	C1	C2	C3	C4	C5	Expected query / action
R25	Y	Y	N	N	N	WHERE name LIKE '%kw%' AND category = 'X'
R26	Y	Y	N	N	Y	WHERE name LIKE '%kw%' AND category = 'X' AND stock > 0
R27	Y	Y	N	Y	N	WHERE name LIKE '%kw%' AND category = 'X' AND brand IN (b1, b2)
R28	Y	Y	N	Y	Y	WHERE name LIKE '%kw%' AND category = 'X' AND brand IN (b1, b2) AND stock > 0
R29	Y	Y	Y	N	N	WHERE name LIKE '%kw%' AND category = 'X' AND price BETWEEN min AND max
R30	Y	Y	Y	N	Y	WHERE name LIKE '%kw%' AND category = 'X' AND price BETWEEN min AND max AND stock > 0
R31	Y	Y	Y	Y	N	WHERE name LIKE '%kw%' AND category = 'X' AND price BETWEEN min AND max AND brand IN (b1, b2)
R32	Y	Y	Y	Y	Y	WHERE name LIKE '%kw%' AND category = 'X' AND price BETWEEN min AND max AND brand IN (b1, b2) AND stock > 0

10.5 Additional Tests Beyond the 5-Condition Table

TC	Scenario	Expected
X1	Brand include (Dell) AND exclude (Dell)	Empty result set; no error
X2	Price min > max	Validation message (should not silently return zero rows)
X3	Price min = max; min = 0; negative price	Exact price match; accepted; rejected
X4	Keyword with %, _ , quote or SQL string	Treated as literal; no injection (parameterized query)
X5	Two categories selected (OR) plus price	(cat1 OR cat2) AND price: brackets applied correctly
X6	All filters applied and zero matches	"No results" page
X7	Result count > page size (20)	Pagination correct; no repeated or missing items
X8	Sort order with each filter set	Order is stable and correct

10.6 Evaluation of Completeness

Aspect	Assessment
Condition combinations	All 32 combinations of the five filters covered (100%)
Logic operators	AND across filters, OR within multi-select and NOT exclusion covered by rules plus X1 and X5
Value-level coverage	The table treats each filter as present or absent; the actual values (min/max, multiple brands) need EP and BVA, partially covered by X2, X3 and X5
Limitations	Sort order, pagination, relevance ranking and performance on a large catalog are not part of the table; they are covered by X7, X8 and separate performance tests

Aspect	Assessment
Reduction option	If combinations are too many, rules can be merged using pairwise testing without losing the pairs of conditions

Conclusion: the reconstructed decision table is complete for the inferred filter logic, and the extra tests close the gaps in value handling, invalid input and security, so the search and filter module is covered both structurally and functionally.

Rubrics for Calculation: 3

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Clear and in-depth understanding of the system and its functionality	Good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation